

Module 4: Supervised Learning
Submodule: 4.3 Newton's Method
Lecture: Newton's Method

XCS229 Machine Learning

[Nandita Bhaskhar]

Contents

I	Introduction	1
II	Finding Solutions	1
III	How Newton's Method Works	2
	Convergence of Newton's Method	3
IV	Generalizing Newton's method	3
V	Rough comparison of Optimization Methods	4
VI	Questions	5
VII	Summary	6

I. Introduction

In this lecture, we will cover the **Newton's method**. As we build up our vocabulary, there have been phenomenal questions about why use batch gradient descent and why stochastic gradient descent. Newton's method is a powerful and popular closed-form solution to the optimization problem. It was a dominant form of solving these models for a long time. This method is not only interesting but also incredibly beautiful and historical.

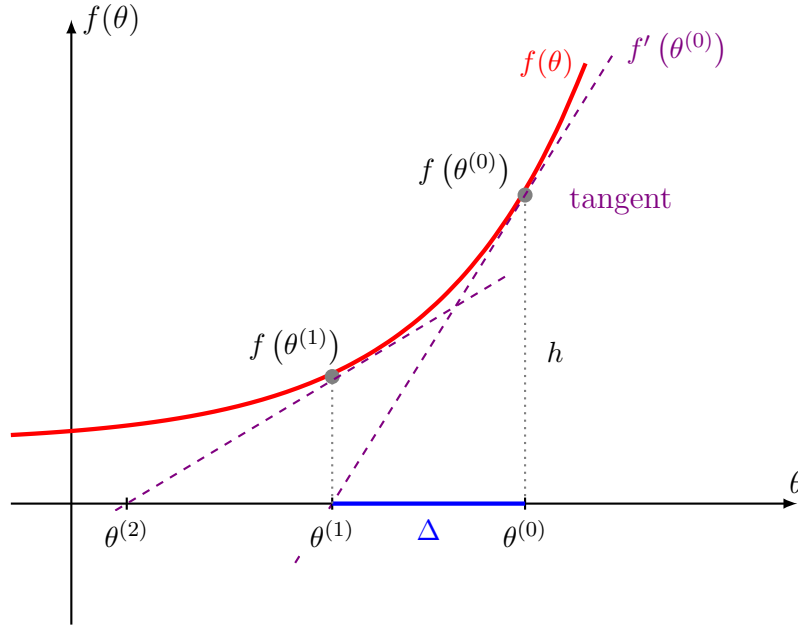
II. Finding Solutions

Our problem is that we are given a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$. For illustration purposes, we will look at the simple case of $d = 1$. Our goal is to find x such that $f(x) = 0$.

This is related to machine learning because when we seek to maximize a likelihood function (e.g., $\arg \max_{\theta} \ell(\theta)$), the first-order optimality condition tells us that $\ell'(\theta) = 0$. Therefore, finding the roots of derivatives relates to maximizing or minimizing functions.

III. How Newton's Method Works

We will explore how this method works with a simple function.



Newton's method is an iterative process similar to other methods we have seen before. The following steps elaborate on how it functions:

- Start with an initial guess $\theta^{(0)}$. This is where the method begins.
- Calculate $f(\theta^{(0)})$ to find the value of the function at this point.
- Compute the derivative $f'(\theta^{(0)})$, which represents the slope of the function at $\theta^{(0)}$.
- Newton's method then states that we will follow this line until it intersects the zero point. This will provide us with our next guess. We define this next guess as $\theta^{(1)} = \theta^{(0)} - \Delta$, where Δ represents the distance to be subtracted.
- Calculate the distance Δ based on the idea of rise over run: $f(\theta^{(0)}) = f'(\theta^{(0)}) \cdot \Delta$. This represents the concept that the function value at $\theta^{(0)}$ is equal to the slope at that point multiplied by the distance Δ . Rearranging, this gives us $\Delta = \frac{f(\theta^{(0)})}{f'(\theta^{(0)})}$.

This last step leads us to understand Newton’s method. Thus,

$$\theta^{(1)} = \theta^{(0)} - \frac{f(\theta^{(0)})}{f'(\theta^{(0)})}.$$

Newtons method performs the following update:

$$\theta^{(t+1)} = \theta^{(t)} - \frac{f(\theta^{(t)})}{f'(\theta^{(t)})}.$$

Convergence of Newton’s Method

- This method converges locally, meaning it will ultimately reach a root (zero) if one exists.
- The convergence rate is astonishingly fast; it exhibits quadratic convergence. If the first estimate after time T is 0.1, the next step might be 0.01, and even further down to 0.0001. The number of significant digits typically doubles with each iteration:

Numbers: $0.1 \rightarrow 0.01 \rightarrow 0.0001$

- Therefore, the error reduces very rapidly. Each step gets you closer to the solution more quickly.
- However, despite its advantages, there are reasons that we don’t use Newton’s method everywhere: the computation can be quite challenging and resource-intensive.

IV. Generalizing Newton’s method

In our logistic regression setting, the parameter vector θ is multidimensional. Thus, we need to generalize Newton’s method for this case. The generalized version of Newton’s method, often referred to as the **Newton-Raphson method**, is defined by the following equation:

$$\theta := \theta - H^{-1} \nabla_{\theta} \ell(\theta).$$

Where:

- $\nabla_{\theta} \ell(\theta)$ is the vector of partial derivatives of the log-likelihood function $\ell(\theta)$ with respect to the components of the parameter vector θ .
- H is a $(d+1) \times (d+1)$ matrix known as the **Hessian**. This matrix is composed of second-order partial derivatives, and its entries are given by:

$$H_{ij} = \frac{\partial^2 \ell(\theta)}{\partial \theta_i \partial \theta_j} \in \mathbb{R}^{(d+1) \times (d+1)}$$

In summary, the method updates the parameter vector θ by subtracting the product of the inverse of the Hessian and the gradient of the log-likelihood, allowing for efficient optimization in a multidimensional parameter space.

V. Rough comparison of Optimization Methods

In this section, we will clarify the differences between various optimization methods and discuss the rationale behind selecting each one. It is crucial to understand how these methods operate in order to conduct a meaningful comparison. This discussion goes beyond mere technical details; it represents a conceptual shift that highlights the intersection of traditional optimization techniques with the evolving landscape of machine learning. By grasping these distinctions, we can make informed choices about which methods are best suited to address our current challenges effectively.

Method	Per iteration	Compute	Steps to error ϵ
SGD	1 data point	$O(d)$	ϵ^{-2}
Batch SGD	n data points	$O(nd)$	$\approx \epsilon^{-1}$
Newton's	n data points	$O(nd^2)$	$\log\left(\frac{1}{\epsilon}\right)$

- **Stochastic Gradient Descent (SGD):** SGD takes an individual data point or a small mini-batch at each iteration. The computation required is proportional to the dimension of the problem, resulting in $O(d)$ complexity. It also requires approximately ϵ^{-2} iterations to reach a given error tolerance ϵ .
- **Batch Stochastic Gradient Descent:** Batch SGD computes on all data points, requiring calculations proportional to the number of data points n and the dimensionality d , giving it a complexity of $O(nd)$. It converges a bit more slowly than SGD, requiring about ϵ^{-1} iterations.
- **Newton's Method:** Newton's Method also considers all n data points but it has a complexity of $O(nd^2)$ because it computes the Hessian. It converges logarithmically, taking only $\log\left(\frac{1}{\epsilon}\right)$ steps to reach the error tolerance ϵ . This method is exceptionally fast given that it provides significant digits with each iteration.

These differences matter significantly in the context of classical statistics versus modern machine learning. In classical statistics, n and d are typically small, making batch methods quite effective. However, in machine learning,

both n and d are often extremely large. For example, models like GPT-3 have 175 billion parameters, which makes the computation of the Hessian (requiring roughly 175 billion squared computations) practically impossible for methods like Newton's.

When training large models, modern strategies limit the number of passes over the data (often tens rather than millions), making SGD a better choice when n is large.

In summary, while traditional methods like Newton's provide exact answers and fast convergence under ideal circumstances, SGD and its stochastic nature become more relevant in high-dimensional and large-dataset scenarios, as is common in machine learning today.

VI. Questions

- **Question:** How can we binary classify time series data?

Answer: Binary classification involves categorizing data into two distinct categories (e.g., yes/no). If the target variable (Y) has more than two classes, a different prediction approach is needed. This session focuses on binary classifications, not multiclass scenarios (like identifying a cat, dog, car).

- Gradient and Hessian
 - The gradient is a vector that represents the direction of the steepest ascent of a function.
 - The Hessian is the second derivative of the loss function, giving us information about the curvature of the function.
 - For a loss function, the Hessian is a matrix derived from the gradient, providing insights into how the loss changes near a point in a multidimensional space.
- Multiclass Classification
 - Multiclass classification goes beyond binary scenarios (like cat, dog, horse, car) and will be discussed in greater detail in the upcoming lecture.
 - Logistic regression can be extended to handle multiple classes through strategies like “one versus all,” which breaks down a multiclass problem into multiple binary problems.
- Generalizing Functions and Newton's Method
 - Newton's method involves finding roots of functions. When applied to machine learning, it simplifies the minimization process of loss functions.

- By generalizing to multiple $(d + 1)$ dimensions, we incorporate matrix representations for derivatives, maintaining the relationships of gradients and Hessians.
- Probabilistic Interpretations
 - A theme in the course is utilizing a probabilistic interpretation to derive models like linear regression and logistic regression.
 - This approach helps unify the understanding of classification and regression tasks, indicating that while they may appear distinct at first, they share fundamental underlying mechanisms.
- Convergence and Optimization
 - Newton’s Method can find local minima, which raises concerns about potentially missing global minima. The challenge exists due to the nature of the functions. Convex functions guarantee that a local minimum is also global.
 - Modern machine learning often deals with non-convex functions, complicating the optimization process.
- Modeling Error Distributions
 - Beyond Gaussians, various distributions can model errors effectively. The upcoming lectures will cover exponential family distributions, showcasing their relevance across various modeling scenarios.
 - Understanding different assumptions about model errors will be critical throughout the course, especially as it applies to supervised and unsupervised learning.

VII. Summary

In this lecture on Newton’s Method, we’ve covered several important concepts:

- **Newton’s Method:** A powerful iterative technique for finding roots of real-valued functions which is particularly useful in optimization problems within machine learning.
- **Convergence Rate:** Newton’s Method exhibits quadratic convergence, meaning that the number of correct digits roughly doubles with each iteration, leading to rapid error reduction.

- **Generalization:** We extended Newton's Method to multidimensional spaces using the Newton-Raphson method. The parameter vector is updated by subtracting the product of the inverse Hessian matrix and the gradient of the log-likelihood function.
- **Comparison of Optimization Methods:** We discussed the differences between Stochastic Gradient Descent (SGD), Batch Stochastic Gradient Descent, and Newton's Method, highlighting their computational complexities and efficiencies under different scenarios.
- **Importance of Selection:** In high-dimensional or large dataset contexts, SGD's efficiency makes it preferable, while Newton's provides significant advantages in smaller, more manageable problems.
- **Connection to Machine Learning:** The principles imbued in these optimization methods serve as fundamental tools that enhance both classical statistics and modern machine learning practices, facilitating better model training and prediction accuracy.